ITDLUPanel - Presentation Documentation

Ääº¡i há»c ÄÃ Láº¡t | Â© 2026

PHáº¦N 1: GIá»šI THIá»†U CHUNG (v1.0)

ITDLUPanel - Báº£ng Äiá»u Khiá»fn Quáº£n LÃ½ MÃ¡y Chá»§ Linux

Ääº¡i há»c ÄÃ Láº¡t | Â© 2026

Slide 1: Giá»›i Thiá»‡u ITDLUPanel

ITDLUPanel

Báº£ng Äiá»u Khiá»fn Quáº£n LÃ½ MÃ¡y Chá»§ Linux

Single Binary â€¢ Zero Dependencies â€¢ Production-Ready

ÄÆ°á»£c phÃ¡t triá»fn bá»Ÿi **Ääº¡i há»c ÄÃ Láº¡t**

Slide 2: Tá»•ng Quan

ITDLUPanel LÃ GÃ¬?

ITDLUPanel (IT Ääº¡i há»c ÄÃ Láº¡t Panel) lÃ má»™t **báº£ng Ä‘iá»u khiá»fn quáº£n lÃ½ mÃ¡y chá»§ Linux hiá»‡n Ä‘áº¡i**, Ä‘Ã³ng gÃ³i trong má»™t file nhá»‹ phÃ¢n duy nháº¥t (~60MB) vá»›i **khÃ´ng phá»¥ thuá»™c runtime bÃªn ngoÃ i**.

Äiá»fm Ná»•i Báºt

- **Single Binary**: ToÃn bá»™ stack trong má»™t file Go
- **Zero Dependencies**: KhÃ´ng cáº§n cÃ i Ä‘áº·t Node.js, Python hay database riÃªng
- **Cross-Platform**: BiÃªn dá»‹ch khÃ´ng cáº§n CGO, cháº¡y trÃªn má»i Linux
- **Production-Ready**: ÄÃ£ Ä‘Æ°á»£c kiá»fm chá»©ng tháº¡i mÃ´i trÆ°á»ng thá»±c táº¿

Slide 3: Táº¡i Sao Chá»n ITDLUPanel?

VÃ¬ Sao Chá»n ITDLUPanel?

So SÃ¡nh Vá»›i CÃ¡c Giáº£i PhÃ¡p KhÃ¡c

TiÃªu ChÃ	ITDLUPanel	cPanel	DirectAdmin
Chi phÃ	Miá»…n phÃ	TrÃº phÃ	TrÃº phÃ
Dependencies	KhÃ´ng	Cáº§n nhiá»u	Cáº§n nhiá»u
Giao diá»‡n	Hiá»‡n Ä‘áº¡i (React)	CÅ©	Trung bÃ¬nh
Docker tÃch há»£p	âœ…	âŒ	âŒ
WebSocket Terminal	âœ…	âŒ	âŒ

PhÃ¹ Há»£p Cho

- **TrÆ°á»ng há»c & Ä‘áº¡i há»c**: Quáº£n lÃ½ server há»c táºp
- **Doanh nghiá»‡p nhá»**: Host website ná»™i bá»™
- **DevOps**: MÃ´i trÆ°á»ng development/staging

Slide 4: Technology Stack

CÃ´ng Nghá»‡ Sá» Dá»¥ng

Backend - Go 1.25

```
â”Œâ”€â”€â”€â
```

```
â”  â””â€â€â€
```

Quáº£n LÃ½ Website ChuyÃªn Nghiá»‡p

Há»— Trá»£ Äa Web Server

```
â”Œâ”€â”€â”€â
```

- ðŸ“ **Compress/Extract**: ZIP, TAR, GZ
- ðŸ” **Permission Editor**: Chmod trá»±c quan
- â **Bookmarks**: ÄÃ¡nh dáº¥u thÆ° má»¥c yÃªu thÃch
- ðŸ” **Search**: TÃ¬m kiáº¿m file nhanh

Supported File Types

```
Languages: JavaScript, TypeScript, Python, Go, PHP,
           Ruby, Java, C++, Rust, HTML, CSS, JSON, YAML...
```

Quick Actions

Action	Description
New File	Táº¡o file má»›i
Upload	Upload file lÃªn server
Download	Táº£i file vá» local
Compress	NÃ©n file/folder
Extract	Giáº£i nÃ©n archive

Slide 11: WebSocket Terminal

Terminal Trong TrÃ¬nh Duyá»‡t

TÃnh NÄƒng Terminal

```
â”Œâ”€â”€â”€â
```

```
â”‚  â””â€”â€”â€
```

Quáº£n LÃ½ Logs ThÃ´ng Minh

Tá»•ng Quan Logs

```
â”Œâ”€â”€â”€â
```

Quáº£n LÃ½ SSL Certificate

SSL Features

```
â”Œâ”€â”€â”€â
```

Slide 17: Internationalization

Äa NgÃ´n Ngá»¯

Supported Languages

```
â”Œâ”€â”€â”€â
```

Slide 19: Deployment

Triá»fn Khai Dá»… DÃ ng

Quick Start

```
# 1. Build
cd web && npm install && npx vite build && cd ..
make build

# 2. Configure
cp config/panel.json.example config/panel.json
vim config/panel.json

# 3. Run
./bin/itdlupanel -config config/panel.json

# 4. Access
# Open https://localhost:8888
```

Default Credentials

Field	Value
Username	`admin`
Password	`admin@123456`
Default Port	`8888`

System Requirements

Resource	Minimum	Recommended
RAM	1 GB	2 GB
CPU	1 core	2+ cores
Disk	5 GB	20 GB
OS	Ubuntu 18.04+	Ubuntu 22.04 LTS

Slide 20: Roadmap

Lá»™ TrÃ¬nh PhÃ¡t Triá»fn

Phase 1: âœ… Completed

- [x] Core panel functionality
- [x] Docker management
- [x] WebSocket terminal
- [x] SSL certificates
- [x] Multi-language support

Phase 2: ðŸš§ In Progress

- [ ] Kubernetes cluster management
- [ ] Advanced monitoring (Prometheus/Grafana)
- [ ] Plugin marketplace
- [ ] API documentation (OpenAPI)

Phase 3: ðŸ“‹ Planned

- [ ] Multi-server management
- [ ] Mobile app (React Native)
- [ ] AI-powered suggestions

- [ ] GitOps integration

Slide 21: KÁº¿t Luáºn

CÁº£m Æ n!

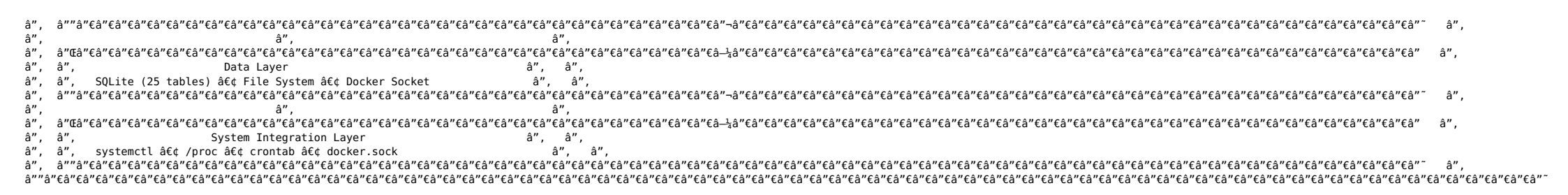
ITDLUPanel

Contact & Support

- ðŸŒ **Website**: https://github.com/itdlu/ITDLUPanel
- ðŸ“§ **Email**: support@itdlu.edu.vn
- ðŸ“– **Docs**: https://docs.itdlu.edu.vn

Key Takeaways

```
â”Œâ”€â”€â”€â
```



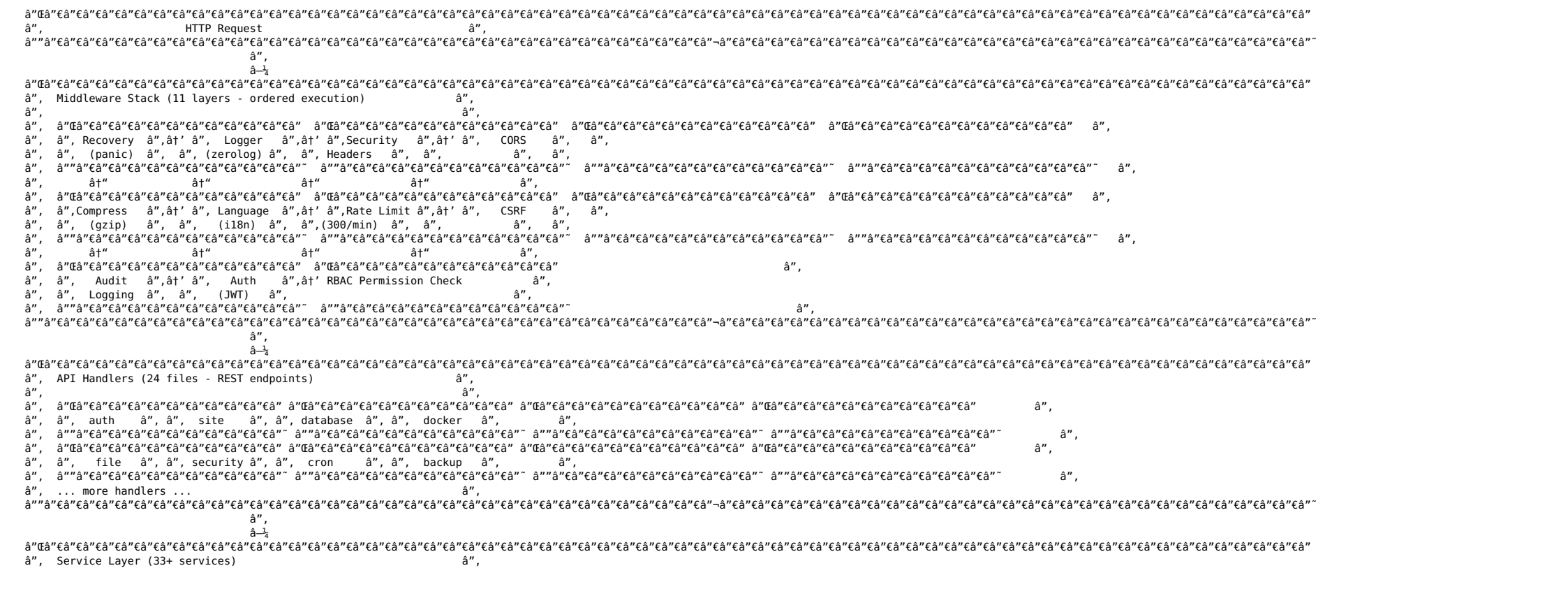
Key Metrics

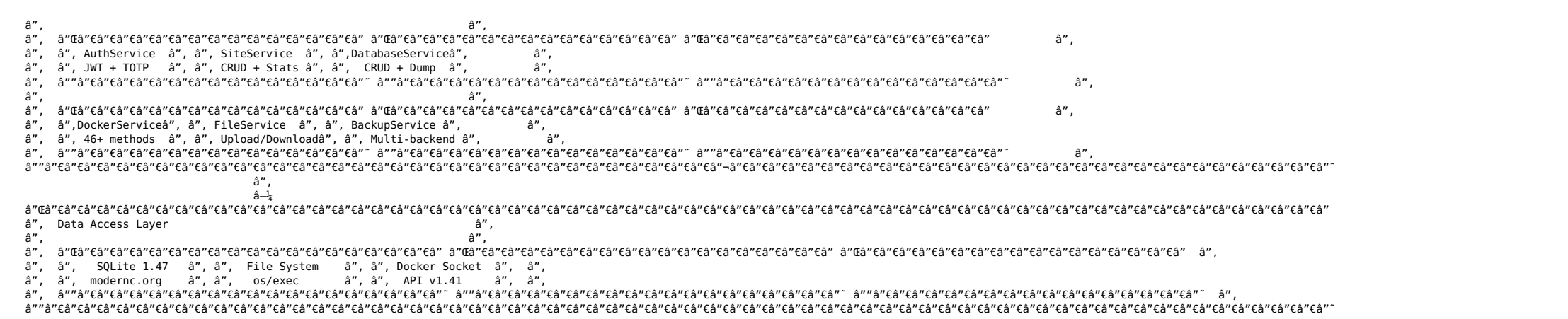
Metric	Value
Go Files	112 files
Lines of Code	43,314 LOC
API Endpoints	~160 REST + 2 WS
Database Tables	25
Services	33+
Middleware Layers	11

Slide 2: Backend Layer Architecture

Backend Go Architecture

Layered Design Pattern





Slide 3: Middleware Stack Chi Tiáº¿t

Middleware Pipeline Deep Dive

11-Layer Middleware Stack

Layer 1: Recovery Middleware

```
func RecoveryMiddleware() gin.HandlerFunc {
    return func(c *gin.Context) {
        defer func() {
            if err := recover(); err != nil {
                // Log panic with stack trace
                zerolog.Error().
                    Interface("panic", err).
                    Str("stack", string(debug.Stack())).
                    Msg("Panic recovered")

                c.AbortWithStatusJSON(500, gin.H{
                    "error": "Internal server error",
                })
            }
        }()
        c.Next()
    }
}
```

Layer 2: Structured Logging

```
// zerolog with lumberjack rotation
zerolog.TimeFieldFormat = zerolog.TimeFormatUnix
log := zerolog.New(os.Stdout).
    Level(zerolog.InfoLevel).
    With().
    Timestamp().
    Caller().
    Logger()
```

Layer 3-6: Security Layers

Layer	Header	Purpose
Security	X-Frame-Options	Clickjacking protection
Security	X-Content-Type-Options	MIME sniffing prevention
Security	X-XSS-Protection	XSS filter (legacy)
Security	Strict-Transport-Security	Force HTTPS
CORS	Access-Control-*	Cross-origin requests

Layer 7: Rate Limiting

```
Algorithm: Token Bucket
â”œâ”€â”€ Rate: 300 requests/minute
â”œâ”€â”€ Per: IP address
â””â”€â”€ Response: 429 Too Many Requests
```

Layer 8: CSRF Protection

```
â”Œâ”€â”€â”€â
```

```
â”‚   â”‚   â”œâ”€â”€ backup.go          # Backup management
â”‚   â”‚   â”œâ”€â”€ security.go         # Firewall, SSH, Fail2ban
â”‚   â”‚   â”œâ”€â”€ cron.go            # Cron jobs
â”‚   â”‚   â”œâ”€â”€ log.go             # Log viewer
â”‚   â”‚   â”œâ”€â”€ monitor.go         # System metrics
â”‚   â”‚   â”œâ”€â”€ user.go            # User management
â”‚   â”‚   â”œâ”€â”€ role.go             # RBAC roles
â”‚   â”‚   â”œâ”€â”€ settings.go         # Panel settings
â”‚   â”‚   â”œâ”€â”€ software.go         # Software installation
â”‚   â”‚   â”œâ”€â”€ ftp.go              # FTP accounts
â”‚   â”‚   â”œâ”€â”€ ftp_storage.go     # FTP backup storage
â”‚   â”‚   â”œâ”€â”€ cloud_storage.go    # S3/GDrive/OneDrive
â”‚   â”‚   â”œâ”€â”€ redis.go           # Redis management
â”‚   â”‚   â”œâ”€â”€ task.go            # Background tasks
â”‚   â”‚   â”œâ”€â”€ notification.go     # Notifications
â”‚   â”‚   â”œâ”€â”€ plugin.go          # Plugin system
â”‚   â”‚   â”œâ”€â”€ waf.go             # WAF management
â”‚   â”‚   â”œâ”€â”€ system.go          # System info
â”‚   â”‚   â””â”€â”€ docker_cleanup.go   # Docker auto-cleanup
â”‚   â”‚
â”‚   â”œâ”€â”€ ws/                    # 2 WebSocket handlers
â”‚   â”‚   â”œâ”€â”€ terminal.go         # PTY terminal (xterm.js â†” creack/pty)
â”‚   â”‚   â””â”€â”€ monitor.go          # Real-time metrics stream
â”‚
â”œâ”€â”€ app/                       # Application bootstrap
â”‚   â”œâ”€â”€ app.go                  # Main entry, graceful shutdown
â”‚   â”œâ”€â”€ router.go               # Gin route registration
â”‚   â”œâ”€â”€ auth_middleware.go      # JWT + RBAC middleware
â”‚   â””â”€â”€ middleware.go          # CORS, rate limit, compression
â”‚
â”œâ”€â”€ service/                   # Business logic (23,437 LOC)
â”‚   â”œâ”€â”€ auth_service.go         # Login, register, 2FA
â”‚   â”œâ”€â”€ jwt.go                  # Token generation/validation
â”‚   â”œâ”€â”€ totp_service.go         # TOTP 2FA (RFC 6238)
â”‚   â”œâ”€â”€ user_service.go         # User CRUD
â”‚   â”œâ”€â”€ role_service.go         # RBAC role management
â”‚   â”œâ”€â”€ site_service.go         # Website CRUD + config
â”‚   â”œâ”€â”€ site_statistics_service.go  # Traffic statistics
â”‚   â”œâ”€â”€ database_service.go     # MySQL/MariaDB management
â”‚   â”œâ”€â”€ file_service.go         # File operations
â”‚   â”œâ”€â”€ docker_service.go       # Docker API wrapper
â”‚   â”œâ”€â”€ docker_compose_service.go   # Compose project management
â”‚   â”œâ”€â”€ docker_cleanup_service.go    # Auto-cleanup old images/containers
â”‚   â”œâ”€â”€ backup_service.go       # Backup orchestration
â”‚   â”œâ”€â”€ backup_incremental.go    # Incremental backup logic
â”‚   â”œâ”€â”€ backup_encryption.go    # AES-256 encryption
â”‚   â”œâ”€â”€ backup_upload_service.go    # Multi-destination upload
â”‚   â”œâ”€â”€ ssl_service.go          # Let's Encrypt integration
â”‚   â”œâ”€â”€ ssl_renewer.go          # Auto-renewal scheduler
â”‚   â”œâ”€â”€ cron_service.go         # Cron job management
â”‚   â”œâ”€â”€ terminal_service.go     # PTY session management
â”‚   â”œâ”€â”€ monitor_service.go      # System metrics collection
â”‚   â”œâ”€â”€ metrics_collector.go    # CPU/RAM/Disk/Network
â”‚   â”œâ”€â”€ security_service.go     # SSH, Fail2ban, hardening
â”‚   â”œâ”€â”€ firewall_service.go     # UFW rules management
â”‚   â”œâ”€â”€ software_service.go     # Software install/uninstall
â”‚   â”œâ”€â”€ settings_service.go     # Panel configuration
â”‚   â”œâ”€â”€ redis_service.go        # Redis management
â”‚   â”œâ”€â”€ ftp_service.go         # PureFTPd management
â”‚   â”œâ”€â”€ ftp_storage_service.go  # FTP storage backends
â”‚   â”œâ”€â”€ cloud_storage_service.go # S3/GDrive/OneDrive
â”‚   â”œâ”€â”€ notification_service.go # Email/Telegram/Webhook
â”‚   â”œâ”€â”€ email_service.go        # SMTP integration
â”‚   â”œâ”€â”€ telegram_service.go     # Telegram bot notifications
â”‚   â”œâ”€â”€ geo_service.go          # GeoIP lookup
â”‚   â”œâ”€â”€ log_service.go         # Log aggregation
â”‚   â”œâ”€â”€ waf_service.go         # WAF rule management
â”‚   â”œâ”€â”€ waf_log_service.go     # WAF access logs
â”‚   â”œâ”€â”€ haproxy_manager.go      # HAProxy configuration
â”‚   â”œâ”€â”€ coraza_spoa.go         # Coraza WAF SPOA integration
â”‚   â”œâ”€â”€ permission_cache.go     # RBAC permission caching
â”‚   â”œâ”€â”€ user_resource_service.go    # User quotas
â”‚   â”œâ”€â”€ site_defaults.go       # Default site configuration
â”‚   â”‚
â”‚   â”œâ”€â”€ installer/             # 14 software installers
â”‚   â”‚   â”œâ”€â”€ installer.go       # Base installer interface
```

```
â”‚   â”‚   â”œâ”€â”€ nginx.go          # Nginx installation
â”‚   â”‚   â”œâ”€â”€ apache.go         # Apache installation
â”‚   â”‚   â”œâ”€â”€ ols.go            # OpenLiteSpeed
â”‚   â”‚   â”œâ”€â”€ php.go            # PHP (multiple versions)
â”‚   â”‚   â”œâ”€â”€ mysql.go          # MySQL installation
â”‚   â”‚   â”œâ”€â”€ postgresql.go     # PostgreSQL
â”‚   â”‚   â”œâ”€â”€ mongodb.go        # MongoDB
â”‚   â”‚   â”œâ”€â”€ redis.go          # Redis
â”‚   â”‚   â”œâ”€â”€ memcached.go      # Memcached
â”‚   â”‚   â”œâ”€â”€ phpmyadmin.go     # phpMyAdmin
â”‚   â”‚   â”œâ”€â”€ pgadmin.go        # pgAdmin
â”‚   â”‚   â”œâ”€â”€ pureftpd.go       # PureFTPd
â”‚   â”‚   â”œâ”€â”€ supervisor.go     # Supervisor
â”‚   â”‚   â”œâ”€â”€ docker.go         # Docker Engine
â”‚   â”‚   â”œâ”€â”€ nodejs.go         # Node.js
â”‚   â”‚   â”œâ”€â”€ python.go         # Python
â”‚   â”‚   â”œâ”€â”€ golang.go         # Go
â”‚   â”‚   â”œâ”€â”€ java.go           # Java/Tomcat
â”‚   â”‚   â”œâ”€â”€ elasticsearch.go  # Elasticsearch
â”‚   â”‚   â”œâ”€â”€ s3_storage.go     # MinIO S3
â”‚   â”‚   â”œâ”€â”€ google_drive.go   # Google Drive OAuth
â”‚   â”‚   â”œâ”€â”€ onedrive.go       # OneDrive OAuth
â”‚   â”‚   â””â”€â”€ ftp_storage.go    # FTP storage
â”‚   â”‚
â”‚   â”œâ”€â”€ vhost/                # 4 web server vhost generators
â”‚   â”‚   â”œâ”€â”€ vhost.go          # Base vhost interface
â”‚   â”‚   â”œâ”€â”€ nginx.go          # Nginx site config
â”‚   â”‚   â”œâ”€â”€ apache.go         # Apache vhost
â”‚   â”‚   â””â”€â”€ ols.go            # OpenLiteSpeed vhost
â”‚   â”‚
â”‚   â”œâ”€â”€ database/             # SQLite + models
â”‚   â”‚   â”œâ”€â”€ sqlite.go         # Database connection pool
â”‚   â”‚   â”œâ”€â”€ models/           # Data models
â”‚   â”‚   â”‚   â””â”€â”€ role.go       # RBAC models
â”‚   â”‚   â”œâ”€â”€ migrations/       # 14 migration files
â”‚   â”‚   â”‚   â”œâ”€â”€ migrations.go # Migration runner
â”‚   â”‚   â”‚   â”œâ”€â”€ 002_monitor_metrics.go
â”‚   â”‚   â”‚   â”œâ”€â”€ 003_firewall_settings.go
â”‚   â”‚   â”‚   â”œâ”€â”€ 004_phpmyadmin_default_port.go
â”‚   â”‚   â”‚   â”œâ”€â”€ 005_quota.go
â”‚   â”‚   â”‚   â”œâ”€â”€ 006_site_description.go
â”‚   â”‚   â”‚   â”œâ”€â”€ 007_permissions.go
â”‚   â”‚   â”‚   â”œâ”€â”€ 008_panel_settings.go
â”‚   â”‚   â”‚   â”œâ”€â”€ 009_ftp_storage.go
â”‚   â”‚   â”‚   â”œâ”€â”€ 010_cloud_storages.go
â”‚   â”‚   â”‚   â”œâ”€â”€ 011_user_resources.go
â”‚   â”‚   â”‚   â”œâ”€â”€ 012_notifications.go
â”‚   â”‚   â”‚   â”œâ”€â”€ 013_notification_templates.go
â”‚   â”‚   â”‚   â”œâ”€â”€ 014_site_type.go
â”‚   â”‚   â”‚   â”œâ”€â”€ 015_docker_cleanup.go
â”‚   â”‚   â”‚   â”œâ”€â”€ 016_waf.go
â”‚   â”‚   â”‚   â”œâ”€â”€ 017_waf_enhance.go
â”‚   â”‚   â”‚   â”œâ”€â”€ 018_backup_uploads.go
â”‚   â”‚   â”‚   â”œâ”€â”€ 019_cron_storage.go
â”‚   â”‚   â”‚   â””â”€â”€ 020_incremental_backups.go
â”‚   â”‚
â”‚   â”œâ”€â”€ config/               # Configuration
â”‚   â”‚   â””â”€â”€ config.go         # JSON config reader/writer
â”‚   â”‚
â”‚   â”œâ”€â”€ pkg/                  # Shared utilities
â”‚   â”‚   â””â”€â”€ ...               # Crypto, logger, errors
â”‚   â”‚
â”‚   â”œâ”€â”€ i18n/                 # Backend i18n
â”‚   â”‚   â””â”€â”€ ...               # EN/VI translations
â”‚   â”‚
â”‚   â”œâ”€â”€ task/                 # Background task queue
â”‚   â”‚   â””â”€â”€ queue.go          # Async task with progress
â”‚   â”‚
â”‚   â”œâ”€â”€ websocket/            # WebSocket hub
â”‚   â”‚   â””â”€â”€ ...               # Connection management
â”‚   â”‚
â”‚   â”œâ”€â”€ system/               # System utilities
â”‚   â”‚   â”œâ”€â”€ shell.go          # Shell execution
â”‚   â”‚   â””â”€â”€ os_detect.go      # OS detection
â”‚   â”‚
â”‚   â””â”€â”€ plugin/               # Plugin system
```

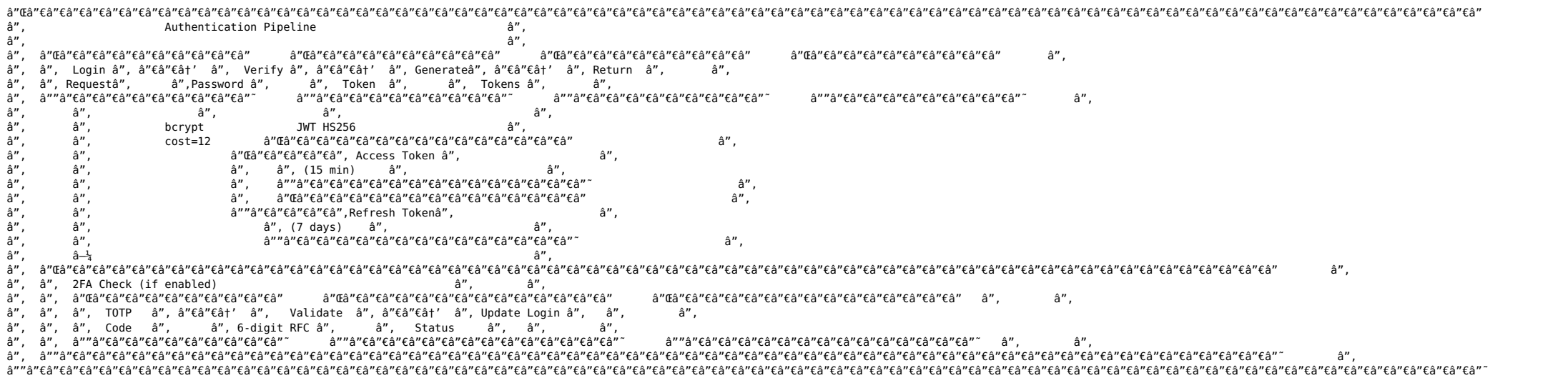
```
    â”œâ”€â”€ plugin.go          # Plugin interface
    â””â”€â”€ manager.go         # Plugin loader

cmd/
â””â”€â”€ panel/
    â””â”€â”€ main.go            # Entry point
```

Slide 5: Authentication & Authorization

JWT + TOTP 2FA + RBAC

Authentication Flow

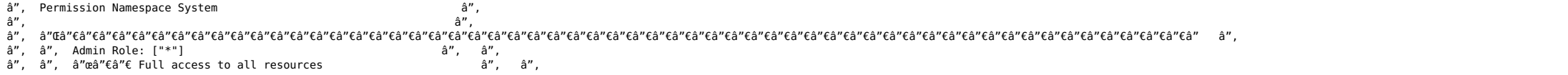


JWT Token Structure

```go
// Access Token Claims (15 minutes TTL)
type AccessClaims struct {
    UserID      uint     `json:"user_id"`
    Username    string   `json:"username"`
    Email       string   `json:"email"`
    Role        string   `json:"role"`
    Permissions []string `json:"permissions"`  // Cached permissions
    TokenType   string   `json:"type"`         // "access"
    RegisteredClaims
}

// Refresh Token Claims (7 days TTL)
type RefreshClaims struct {
    UserID    uint   `json:"user_id"`
    TokenType string `json:"type"`  // "refresh"
    jti       string `json:"jti"`  // Unique token ID for revocation
    RegisteredClaims
}
```

RBAC Permission Model



```
â”‚  â”‚  â””â”€ Can manage users and roles                          â”‚   â”‚
â”‚  â””â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”˜   â”‚
â”‚                                                                 â”‚
â”‚  â”Œâ”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”   â”‚
â”‚  â”‚  Operator Role:                                    â”‚   â”‚
â”‚  â”‚  â”œâ”€ sites.*           (all site operations)           â”‚   â”‚
â”‚  â”‚  â”œâ”€ databases.*       (all database operations)       â”‚   â”‚
â”‚  â”‚  â”œâ”€ files.*           (all file operations)           â”‚   â”‚
â”‚  â”‚  â”œâ”€ docker.*          (all docker operations)         â”‚   â”‚
â”‚  â”‚  â”œâ”€ cron.*            (all cron operations)           â”‚   â”‚
â”‚  â”‚  â”œâ”€ backup.*          (all backup operations)         â”‚   â”‚
â”‚  â”‚  â””â”€ terminal.access   (terminal access)               â”‚   â”‚
â”‚  â””â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”˜   â”‚
â”‚                                                                 â”‚
â”‚  â”Œâ”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”   â”‚
â”‚  â”‚  Viewer Role:                                      â”‚   â”‚
â”‚  â”‚  â”œâ”€ sites.read         (read-only site view)           â”‚   â”‚
â”‚  â”‚  â”œâ”€ databases.read     (read-only database view)       â”‚   â”‚
â”‚  â”‚  â”œâ”€ files.read         (read-only file view)           â”‚   â”‚
â”‚  â”‚  â”œâ”€ logs.read          (read-only log view)            â”‚   â”‚
â”‚  â”‚  â””â”€ docker.read        (read-only docker view)         â”‚   â”‚
â”‚  â””â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”˜   â”‚
â”‚                                                                 â”‚
â””â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”˜
```

TOTP 2FA Implementation

```
// TOTP Configuration (RFC 6238)
const (
    TOTPIssuer      = "ITDLUPanel"
    TOTPDigits      = 6
    TOTPPeriod      = 30  // seconds
    TOTPSecretSize  = 20  // bytes
)

// Enable 2FA - Generate QR Code Secret
func (s *AuthService) EnableTOTP(userID uint) (*TOTPResponse, error) {
    // Generate random secret
    secret, err := totp.Generate(totp.GenerateOpts{
        Issuer:      TOTPIssuer,
        AccountName: username,
        SecretSize:  TOTPSecretSize,
        Period:      TOTPPeriod,
        Digits:      otp.DigitsSix,
        Algorithm:   otp.AlgorithmSHA1,
    })

    // Generate provisioning URI for QR code
    provisioningURI := secret.URL()

    return &TOTPResponse{
        Secret:     secret.Secret(),
        ProvisioningURI: provisioningURI,
        QRCode:     generateQRCodeBase64(provisioningURI),
    }
}

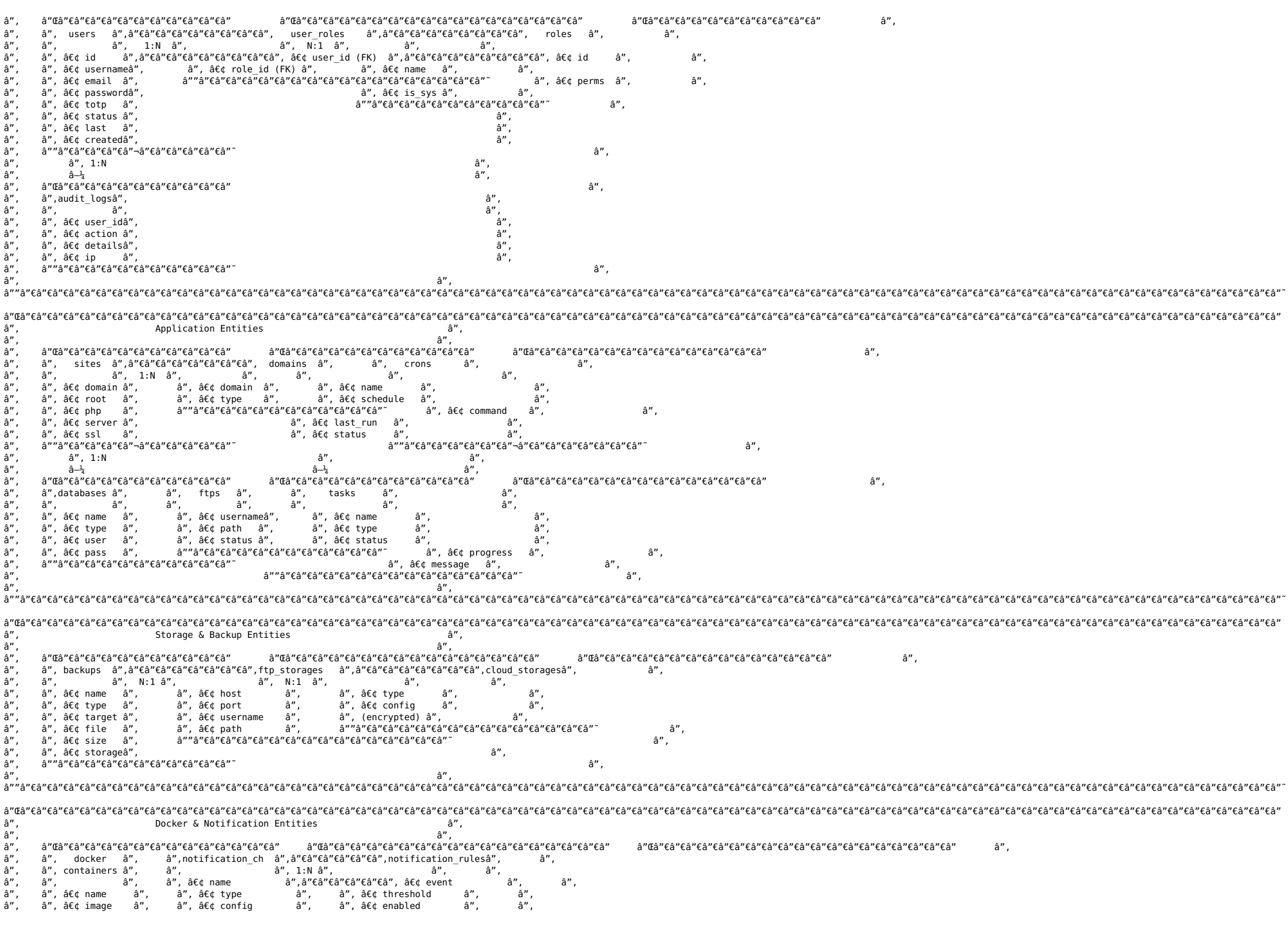
// Verify TOTP Code
func (s *AuthService) VerifyTOTP(secret, code string) bool {
    return totp.Validate(code, secret)
}
```

Slide 6: Database Schema Chi Tiáº¿t

SQLite Database Architecture

Entity Relationship Diagram

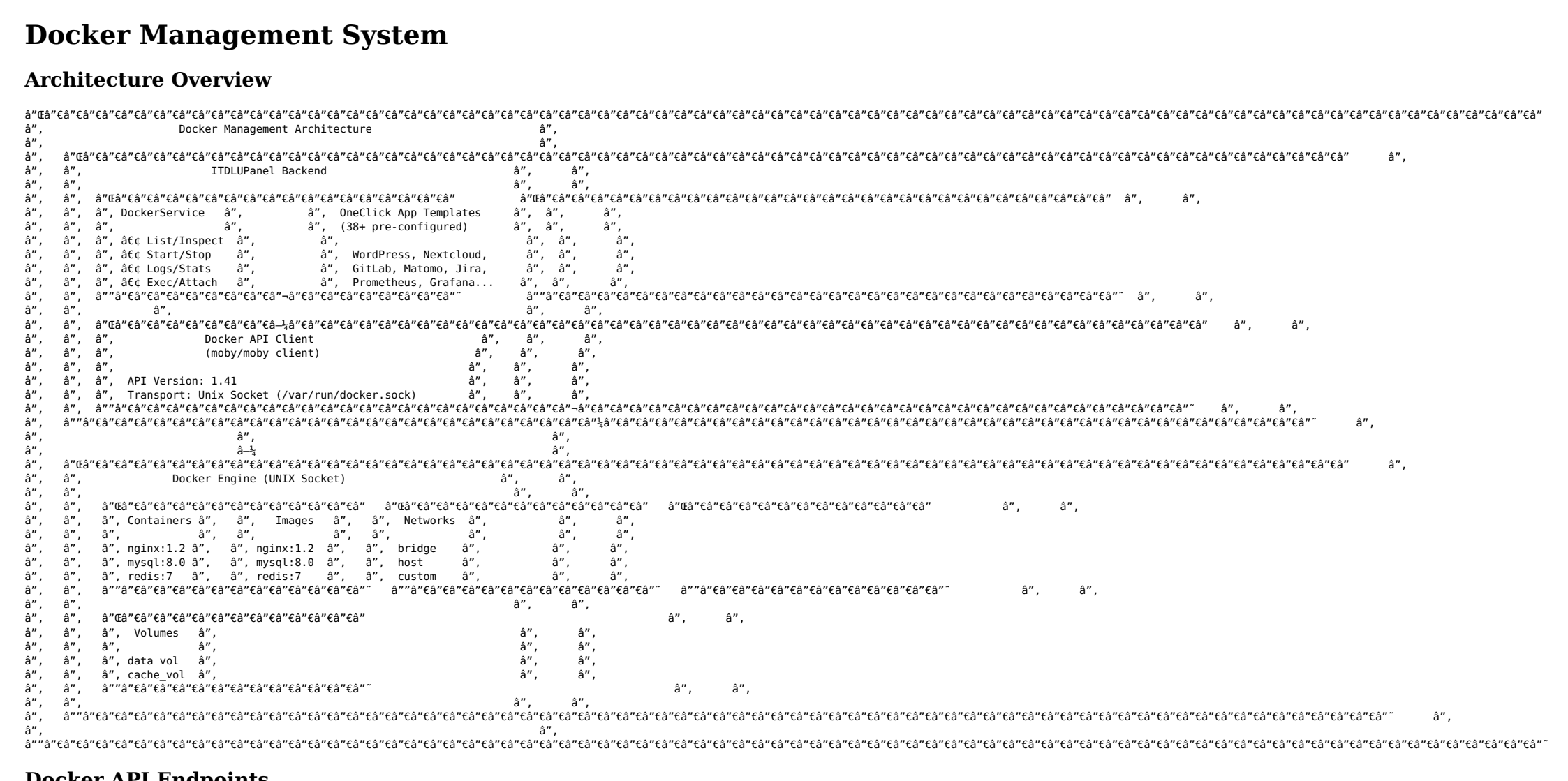
```
â”Œâ”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â
```


users
user_roles
roles
1:N
N:1
id
user_id (FK)
username
role_id (FK)
name
email
perms
password
is_sys
totp
status
last
created
1:N
Application Entities
sites
domains
crons
1:N
domain
type
schedule
php
command
server
last_run
ssl
status
1:N
databases
ftps
tasks
name
username
name
path
type
user
status
pass
progress
message
Storage & Backup Entities
backups
ftp_storages
cloud_storages
N:1
N:1
name
host
type
type
port
config
target
username
(encrypted)
file
path
size
storage
Docker & Notification Entities
docker
notification_ch
notification_rules
containers
1:N
name
event
name
type
threshold
image
config
enabled

```
â”‚    â”‚ â€¢ status   â”‚     â”‚ â€¢ enabled      â”‚     â”œâ”€â”€â”€â
```

Docker Management System

Architecture Overview



Docker API Endpoints

Container Management (20+ endpoints)

```
// Container CRUD
GET    /api/v1/docker/containers             // List all containers
POST   /api/v1/docker/containers             // Create container
GET    /api/v1/docker/containers/:id         // Inspect container
DELETE /api/v1/docker/containers/:id         // Remove container

// Container Lifecycle
POST   /api/v1/docker/containers/:id/start     // Start container
POST   /api/v1/docker/containers/:id/stop      // Stop container
POST   /api/v1/docker/containers/:id/restart    // Restart container
POST   /api/v1/docker/containers/:id/pause      // Pause container
POST   /api/v1/docker/containers/:id/unpause    // Unpause container

// Container Operations
GET    /api/v1/docker/containers/:id/logs    // Get container logs
GET    /api/v1/docker/containers/:id/stats   // Get resource stats
POST   /api/v1/docker/containers/:id/rename  // Rename container
```

```
POST   /api/v1/docker/containers/:id/exec    // Execute command
POST   /api/v1/docker/containers/:id/commit  // Create image from container
```

Image Management (6 endpoints)

```
GET    /api/v1/docker/images                 // List all images
POST   /api/v1/docker/images/pull            // Pull image from registry
DELETE /api/v1/docker/images/:id            // Remove image
GET    /api/v1/docker/images/:id            // Inspect image
GET    /api/v1/docker/images/search         // Search Docker Hub
POST   /api/v1/docker/images/:id/tag        // Tag image
```

Network Management (7 endpoints)

```
GET    /api/v1/docker/networks               // List networks
POST   /api/v1/docker/networks             // Create network
DELETE /api/v1/docker/networks/:id          // Remove network
GET    /api/v1/docker/networks/:id          // Inspect network
POST   /api/v1/docker/networks/:id/connect    // Connect container
POST   /api/v1/docker/networks/:id/disconnect // Disconnect container
```

Volume Management (5 endpoints)

```
GET    /api/v1/docker/volumes               // List volumes
POST   /api/v1/docker/volumes              // Create volume
DELETE /api/v1/docker/volumes/:name        // Remove volume
GET    /api/v1/docker/volumes/:name        // Inspect volume
POST   /api/v1/docker/volumes/prune        // Prune unused volumes
```

Docker Compose (5 endpoints)

```
GET    /api/v1/docker/compose               // List compose projects
POST   /api/v1/docker/compose              // Create compose project
POST   /api/v1/docker/compose/:name/up      // Start project
POST   /api/v1/docker/compose/:name/down   // Stop project
DELETE /api/v1/docker/compose/:name        // Remove project
```

System (5 endpoints)

```
GET    /api/v1/docker/info                 // Docker engine info
GET    /api/v1/docker/daemon/config       // Get daemon.json
POST   /api/v1/docker/daemon/config       // Save daemon.json
POST   /api/v1/docker/restart             // Restart Docker
POST   /api/v1/docker/stop                // Stop Docker
```

OneClick App Templates

```
â”Œâ”€â”€â”€â
```

```
â”‚  â”‚ â€¢ OwnCloud              â”‚ â€¢ Rocket.Chat                        â”‚  â”‚
â”‚  â”‚ â€¢ Jellyfin (Media)      â”‚ â€¢ Bitwarden (Password)               â”‚  â”‚
â”‚  â”‚ â€¢ Emby (Media)          â”‚ â€¢ Invoice Ninja                      â”‚  â”‚
â”‚  â”‚ â€¢ Monica (CRM)          â”‚ â€¢ Monica                             â”‚  â”‚
â”‚  â”˜â””â”€â”€â”€
```

```
// Server â†’ Client Messages
type TerminalOutput struct {
    Type string `json:"type"`  // "output"
    Data string `json:"data"`  // Terminal output (base64 or raw)
}

// Backend Implementation
func (h *TerminalHub) HandleWebSocket(c *gin.Context) {
    // Upgrade HTTP to WebSocket
    conn, err := upgrader.Upgrade(c.Writer, c.Request, nil)
    defer conn.Close()

    // Create PTY
    f, tty, err := pty.StartWithSize(cmd, nil)
    defer f.Close()

    // Bidirectional copy
    go func() {
        io.Copy(f, conn)  // Client â†’ PTY
    }()
    io.Copy(conn, f)     // PTY â†’ Client
}
```

Monitor WebSocket Protocol

```
// Server â†’ Client (every 2 seconds)
type MetricsUpdate struct {
    Type     string    `json:"type"`      // "metrics"
    Timestamp time.Time `json:"timestamp"`
    CPU       CPUMetrics  `json:"cpu"`
    Memory    MemMetrics  `json:"memory"`
    Disk      DiskMetrics `json:"disk"`
    Network   NetMetrics  `json:"network"`
}

type CPUMetrics struct {
    Percent   float64   `json:"percent"`   // 0-100
    User      float64   `json:"user"`
    System    float64   `json:"system"`
    Idle      float64   `json:"idle"`
    CoreCount int       `json:"core_count"`
}

type MemMetrics struct {
    Total       uint64  `json:"total"`      // bytes
    Used        uint64  `json:"used"`
    Available   uint64  `json:"available"`
    UsedPercent float64 `json:"used_percent"`
}

type DiskMetrics struct {
    Total       uint64  `json:"total"`
    Used        uint64  `json:"used"`
    Free        uint64  `json:"free"`
    UsedPercent float64 `json:"used_percent"`
}

type NetMetrics struct {
    BytesSent   uint64 `json:"bytes_sent"`
    BytesRecv   uint64 `json:"bytes_recv"`
    PacketsSent uint64 `json:"packets_sent"`
    PacketsRecv uint64 `json:"packets_recv"`
}
```

Slide 9: File Management System

File Manager Architecture

Capabilities Overview

```
â”Œâ”€â”€â”€â
```

```
POST /api/v1/files/rename          // Rename file/directory
POST /api/v1/files/copy            // Copy file/directory
POST /api/v1/files/move            // Move file/directory
POST /api/v1/files/permissions     // Change permissions (chmod)

// Transfer
POST /api/v1/files/upload          // Upload file (multipart)
GET  /api/v1/files/download        // Download file
POST /api/v1/files/remote-download // Download from URL

// Archive
POST /api/v1/files/compress        // Compress to ZIP/TAR/TARGZ
POST /api/v1/files/decompress      // Extract archive

// Search
POST /api/v1/files/search          // Search files
```

Slide 10: Backup System Architecture

Multi-Destination Backup System

Backup Flow

```
â”Œâ”€â”€â”€â
```

```
    --events --all-databases | gzip > backup_full.sql.gz

# Files
tar -czf backup_files.tar.gz /var/www/sites/
```

Incremental Backup

```
// Incremental backup using rsync-style diff
type IncrementalBackup struct {
    BaseBackupID  uint      `db:"base_backup_id"`
    DiffPath      string    `db:"diff_path"`
    ChangesCount  int       `db:"changes_count"`
    ChangesSize   int64     `db:"changes_size"`
    CreatedAt     time.Time `db:"created_at"`
}

// Only backup changed files since last backup
func (s *BackupService) CreateIncremental(ctx context.Context, siteID uint) error {
    lastBackup := s.getLastBackup(siteID)
    lastBackupTime := lastBackup.CreatedAt

    // Find all modified files
    changes, err := s.findChanges(siteID, lastBackupTime)

    // Create diff archive
    for _, change := range changes {
        s.addToDiff(change)
    }

    return nil
}
```

Encryption

```
// AES-256-GCM Encryption
type EncryptedBackup struct {
    Ciphertext   []byte
    Nonce        []byte    // 12 bytes
    Tag          []byte    // 16 bytes
}

// Encrypt backup
func EncryptBackup(data []byte, password string) (*EncryptedBackup, error) {
    key := deriveKey(password)  // PBKDF2 with 100,000 iterations

    block, err := aes.NewCipher(key)
    gcm, err := cipher.NewGCM(block)

    nonce := make([]byte, gcm.NonceSize())
    io.ReadFull(rand.Reader, nonce)

    sealed := gcm.Seal(nonce, nonce, data, nil)

    return &EncryptedBackup{
        Ciphertext: sealed[len(nonce):],
        Nonce:      nonce,
        Tag:        sealed[len(nonce):],
    }, nil
}
```

Slide 11: SSL Certificate Management

Let's Encrypt Integration

Certificate Lifecycle

```
â”Œâ”€â”€â”€â
```


â”   â”, Request â”,â”€â”€â”€â”€â”€â”€â”€â”¬â”, Issue â”,â”€â”€â”€â”€â”€â”€â”€â”¬â”, Install â”,â”€â”€â”€â”€â”€â”€â”€â”¬â”, Renew â”,   â”,
â”   â”, Cert   â”,   â”, Cert   â”,   â”, Cert   â”,   â”, (auto) â”,   â”,
â”   â”"â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”˜   â”"â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”˜   â”"â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”˜   â”"â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”˜   â”,
â”   â”,
â”   â”, â”Œâ”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â”€â

React 19 SPA Architecture

Component Hierarchy



Key React Patterns

State Management

```
// Zustand Store Example
interface AuthState {
  user: User | null;
  accessToken: string | null;
  refreshToken: string | null;
  isAuthenticated: boolean;
```

```
  // Actions
  login: (credentials: LoginCredentials) => Promise<void>;
  logout: () => void;
  refreshTokens: () => Promise<void>;
  updateProfile: (data: Partial<User>) => void;
}

export const useAuthStore = create<AuthState>((set, get) => ({
  user: null,
  accessToken: null,
  refreshToken: null,
  isAuthenticated: false,

  login: async (credentials) => {
    const response = await api.post('/auth/login', credentials);
    const { user, access_token, refresh_token } = response.data;

    set({
      user,
      accessToken: access_token,
      refreshToken: refresh_token,
      isAuthenticated: true
    });

    // Store in localStorage for persistence
    localStorage.setItem('refresh_token', refresh_token);
  },

  logout: () => {
    api.post('/auth/logout');
    set({ user: null, accessToken: null, refreshToken: null, isAuthenticated: false });
    localStorage.removeItem('refresh_token');
  },
}));
```

Theme System

```
// usePalette hook
export function usePalette() {
  const isDark = document.body.getAttribute('data-theme') === 'dark';

  return {
    isDark,
    p: {
      bg: isDark ? '#0D1117' : '#F5F7FA',
      card: isDark ? '#161B22' : '#FFFFFF',
      cardBorder: isDark ? 'rgba(255,255,255,0.1)' : '#E8EDF2',
      text: isDark ? '#E6EDF3' : '#1A1A2E',
      textMuted: isDark ? '#8B949E' : '#64748B',
      headerBg: isDark ? '#21262D' : '#FAFBFC',
      accent: '#1565C0',
      green: '#52C41A',
      red: '#FF4D4F',
      amber: '#FAAD14',
      purple: '#722ED1',
      cyan: '#13C2C2',
    }
  };
}
```

Slide 13: Performance Optimization

Performance & Optimization

Backend Optimizations

Database Connection Pooling

```
// SQLite connection pool settings
type DBConfig struct {
```

```
    MaxOpenConns    int = 25        // Maximum open connections
    MaxIdleConns    int = 5         // Maximum idle connections
    ConnMaxLifetime time.Duration = time.Hour
}

// Prepared statements for frequently used queries
type PreparedStatements struct {
    GetUserByUsername *sql.Stmt
    GetSitesByUser    *sql.Stmt
    GetContainers     *sql.Stmt
    InsertAuditLog    *sql.Stmt
}
```

Concurrent Request Handling

```
// Gin context pool for memory optimization
engine.Use(gin.Logger())
engine.Use(gin.Recovery())

// Custom context pool
engine.Use(func(c *gin.Context) {
    c.Next()
})

// HTTP server settings
s := &http.Server{
    Addr:         ":8888",
    Handler:      engine,
    ReadTimeout:  30 * time.Second,
    WriteTimeout: 30 * time.Second,
    IdleTimeout:  120 * time.Second,
    MaxHeaderBytes: 1 << 20, // 1MB
}
```

Caching Strategy

```
// Permission cache with TTL
type PermissionCache struct {
    mu    sync.RWMutex
    data  map[uint][]string
    timer map[uint]*time.Timer
    ttl   time.Duration
}

func (c *PermissionCache) Get(userID uint) ([]string, bool) {
    c.mu.RLock()
    defer c.mu.RUnlock()
    perms, ok := c.data[userID]
    return perms, ok
}

func (c *PermissionCache) Set(userID uint, perms []string) {
    c.mu.Lock()
    defer c.mu.Unlock()

    c.data[userID] = perms

    // Reset timer
    if c.timer[userID] != nil {
        c.timer[userID].Stop()
    }
    c.timer[userID] = time.AfterFunc(c.ttl, func() {
        c.Delete(userID)
    })
}
```

Frontend Optimizations

Code Splitting

```
// React.lazy for all pages
const Dashboard = lazy(() => import('./pages/Dashboard'));
const Websites = lazy(() => import('./pages/Websites'));
```

```
const Docker = lazy(() => import('./pages/Docker'));
const Files = lazy(() => import('./pages/Files'));

// Route-level splitting
<Suspense fallback={<Spin />}>
  <Routes>
    <Route path="/dashboard" element={<Dashboard />} />
    <Route path="/docker/*" element={<Docker />} />
  </Routes>
</Suspense>
```

Memoization

```
// Heavy component memoization
const HeavyTable = memo(({ data, columns }: TableProps) => {
  return (
    <Table
      dataSource={data}
      columns={useMemo(() => columns, [columns])}
      pagination={false}
    />
  );
});

// Expensive computation memoization
const processedData = useMemo(() => {
  return data.map(item => transformItem(item));
}, [data]);
```

WebSocket Optimization

```
// Debounced metrics updates
const MetricsUpdater = () => {
  const [metrics, setMetrics] = useState<Metrics>(null);

  useEffect(() => {
    let buffer: Metrics[] = [];
    let rafId: number;

    const processBuffer = () => {
      // Take latest from buffer, ignore others
      if (buffer.length > 0) {
        setMetrics(buffer[buffer.length - 1]);
        buffer = [];
      }
      rafId = requestAnimationFrame(processBuffer);
    };

    ws.onmessage = (event) => {
      buffer.push(JSON.parse(event.data));
    };

    rafId = requestAnimationFrame(processBuffer);
    return () => cancelAnimationFrame(rafId);
  }, []);
};
```

Slide 14: Security Implementation

Security Hardening

Security Layers

```
â”Œâ”€â”€â”€â
```

```
â”‚   â”‚  â”œâ”€â”€ HSTS (HTTP Strict Transport Security)                          â”‚ â”‚
â”‚   â”‚  â”œâ”€â”€ Certificate pinning (optional)                                 â”‚ â”‚
â”‚   â”‚  â””â”€â”€ Regular security updates                                       â”‚ â”‚
â”‚   â”‚                                                                     â”‚ â”‚
â”‚   â”‚                                                                     â”‚ â”‚
â”‚   â”œâ”€â”€ Application Layer                                                 â”‚ â”‚
â”‚   â”‚  â”œâ”€â”€ JWT (HS256)  â”‚ â”œâ”€â”€ TOTP 2FA    â”‚ â”œâ”€â”€ bcrypt     â”‚                 â”‚ â”‚
â”‚   â”‚  â”œâ”€â”€ 15min token  â”‚ â”œâ”€â”€ RBAC      â”‚ â”œâ”€â”€ Rate Limit  â”‚ â”œâ”€â”€ CSRF     â”‚                 â”‚ â”‚
â”‚   â”‚  â”œâ”€â”€ RBAC      â”‚ â”œâ”€â”€ RBAC      â”‚ â”œâ”€â”€ Rate Limit  â”‚ â”œâ”€â”€ CSRF     â”‚                 â”‚ â”‚
â”‚   â”‚  â”œâ”€â”€ 11 namespacesâ”‚ â”œâ”€â”€ 300/min/IP   â”‚ â”œâ”€â”€ tokens    â”‚                 â”‚ â”‚
â”‚   â”‚  â”œâ”€â”€ RBAC      â”‚ â”œâ”€â”€ RBAC      â”‚ â”œâ”€â”€ Rate Limit  â”‚ â”œâ”€â”€ CSRF     â”‚                 â”‚ â”‚
â”‚   â”‚                                                                     â”‚ â”‚
â”‚   â”‚                                                                     â”‚ â”‚
â”‚   â”œâ”€â”€ Infrastructure Layer                                              â”‚ â”‚
â”‚   â”‚  â”œâ”€â”€ Firewall (UFW)                                                 â”‚ â”‚
â”‚   â”‚  â”œâ”€â”€ Fail2ban (brute-force protection)                              â”‚ â”‚
â”‚   â”‚  â”œâ”€â”€ AppArmor/SELinux (optional)                                    â”‚ â”‚
â”‚   â”‚  â”œâ”€â”€ Regular security updates                                       â”‚ â”‚
â”‚   â”‚  â””â”€â”€ HSTS (HTTP Strict Transport Security)                          â”‚ â”‚
â”‚   â”‚                                                                     â”‚ â”‚
â””â”€â”€ Certificate pinning (optional)                                        â”‚ â”‚
```

Security Headers

```
// Middleware: SecurityHeaders
func SecurityHeadersMiddleware() gin.HandlerFunc {
    return func(c *gin.Context) {
        // Prevent clickjacking
        c.Header("X-Frame-Options", "DENY")

        // Prevent MIME sniffing
        c.Header("X-Content-Type-Options", "nosniff")

        // XSS filter (legacy but still useful)
        c.Header("X-XSS-Protection", "1; mode=block")

        // Content Security Policy
        c.Header("Content-Security-Policy",
            "default-src 'self'; script-src 'self' 'unsafe-inline'; style-src 'self' 'unsafe-inline';")

        // Referrer Policy
        c.Header("Referrer-Policy", "strict-origin-when-cross-origin")

        c.Next()
    }
}
```

Password Security

```
// bcrypt with cost=12
const bcryptCost = 12

func HashPassword(password string) (string, error) {
    bytes, err := bcrypt.GenerateFromPassword([]byte(password), bcryptCost)
    return string(bytes), err
}

func CheckPassword(password, hash string) bool {
    err := bcrypt.CompareHashAndPassword([]byte(hash), []byte(password))
    return err == nil
}

// Password requirements
const passwordMinLength = 8
const passwordRequireUppercase = true
const passwordRequireLowercase = true
const passwordRequireNumber = true
const passwordRequireSpecial = true
```

Rate Limiting

```
// Token bucket rate limiter
type RateLimiter struct {
    tokens     float64
    maxTokens  float64
    refillRate float64  // tokens per second
    lastRefill time.Time
    mu         sync.Mutex
}

func (rl *RateLimiter) Allow() bool {
    rl.mu.Lock()
    defer rl.mu.Unlock()

    // Refill tokens
    now := time.Now()
    elapsed := now.Sub(rl.lastRefill).Seconds()
    rl.tokens = min(rl.maxTokens, rl.tokens+elapsed*rl.refillRate)
    rl.lastRefill = now

    // Check if request allowed
    if rl.tokens >= 1 {
        rl.tokens--
        return true
    }
    return false
}

// Configuration: 300 requests per minute = 5 requests per second
rateLimiter := &RateLimiter{
    maxTokens:  300,
    refillRate: 5,
}
```

Slide 15: CI/CD & Deployment

Build & Deployment Pipeline

Build Process

```
â”Œâ”€â”€â”€â
```

Deployment Options

1. Direct Binary

```
# Download and install
wget https://github.com/itdlu/ITDLUPanel/releases/latest/itdlupanel
chmod +x itdlupanel
sudo mv itdlupanel /usr/local/bin/

# Create systemd service
sudo tee /etc/systemd/system/itdlupanel.service <<EOF
[Unit]
Description=ITDLUPanel Server
After=network.target

[Service]
Type=simple
User=root
WorkingDirectory=/ITDLUPanel
ExecStart=/usr/local/bin/itdlupanel -config /ITDLUPanel/config/panel.json
Restart=always
RestartSec=10

[Install]
WantedBy=multi-user.target
EOF

# Enable and start
sudo systemctl daemon-reload
sudo systemctl enable itdlupanel
sudo systemctl start itdlupanel
```

2. Docker Deployment

```
# Dockerfile
FROM alpine:3.21

RUN apk add --no-cache ca-certificates tzdata

COPY itdlupanel /usr/local/bin/
RUN chmod +x /usr/local/bin/itdlupanel

EXPOSE 8888

USER 1000

ENTRYPOINT ["itdlupanel"]
CMD ["-config", "/data/panel.json"]

VOLUME ["/data"]
```

```
# Run with Docker
docker run -d \
  --name itdlupanel \
  -p 8888:8888 \
  -v /ITDLUPanel/data:/data \
  -v /var/run/docker.sock:/var/run/docker.sock \
  --restart unless-stopped \
  itdlu/itdlupanel:latest
```

Slide 16: Code Statistics

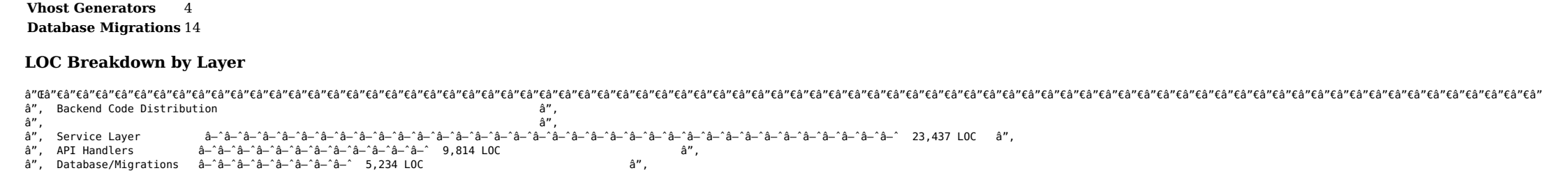
Project Metrics

Backend (Go)

Metric	Count
Total Go Files	112
Total Lines of Code	43,314

API Handlers	24 files
Services	33+
Installers	14
Vhost Generators	4
Database Migrations	14

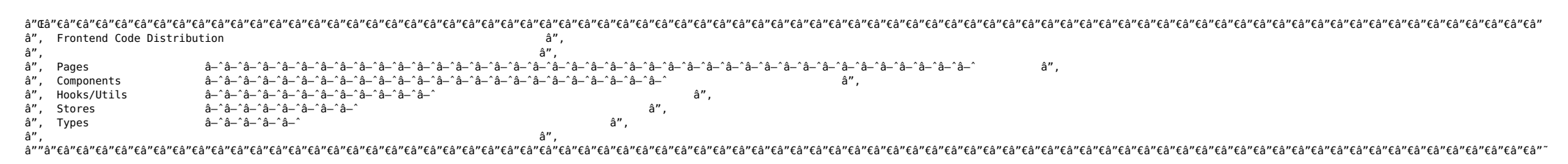
LOC Breakdown by Layer



Frontend (React/TypeScript)

Metric	Count
Total TSX/TS Files	93
Pages	17 (lazy-loaded)
Shared Components	15
Zustand Stores	5
Custom Hooks	7+

Frontend Code Distribution



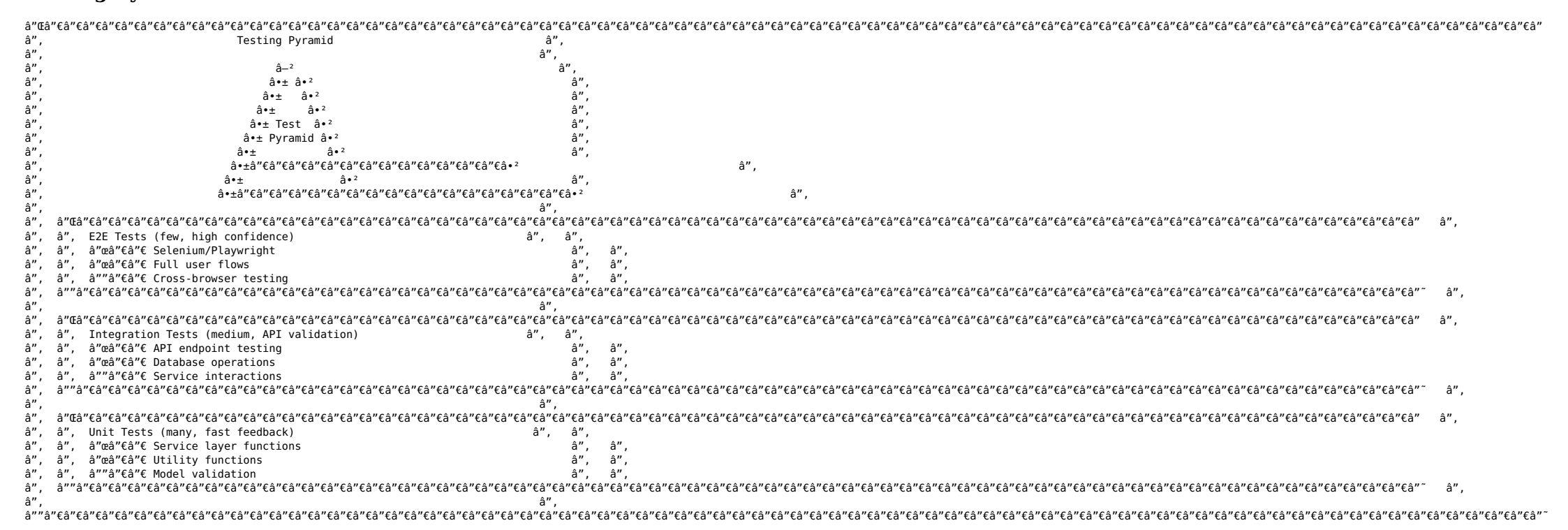
API Coverage

Module	REST Endpoints	WebSocket
Auth	9	-
Dashboard	2	-
Sites	14	-
Databases	8	-
Docker	46+	-
Files	17	-
Security	12	-
Cron	8	-
Logs	7	-
SSL	9	-
Backup	6	-
Monitor	5	1
Terminal	-	1
Total	**~160**	**2**

Slide 17: Testing Strategy

Quality Assurance

Testing Pyramid



Backend Testing

```
// Example: Service unit test
func TestSiteService_Create(t *testing.T) {
    db := setupTestDB(t)
    service := NewSiteService(db)

    // Test successful creation
    site, err := service.Create(&CreateSiteRequest{
        Domain:   "example.com",
        RootPath: "/var/www/example",
        PHPVer:   "8.2",
        WebServ:  "nginx",
    })

    require.NoError(t, err)
    require.NotNil(t, site)
    require.Equal(t, "example.com", site.Domain)

    // Test duplicate domain
    _, err = service.Create(&CreateSiteRequest{
        Domain: "example.com",
    })
    require.Error(t, err)
    require.Equal(t, ErrDomainExists, err)
}

// Example: Integration test
func TestDockerAPI_ListContainers(t *testing.T) {
    if testing.Short() {
        t.Skip("Skipping integration test")
```

```
    }

    // Requires running Docker daemon
    service := NewDockerService()
    containers, err := service.ListContainers()

    require.NoError(t, err)
    require.NotNil(t, containers)
}
```

Frontend Testing

```
// Example: Component test with React Testing Library
describe('SiteCard', () => {
  it('renders site information correctly', () => {
    const site = {
      id: 1,
      domain: 'example.com',
      php_version: '8.2',
      status: 'running'
    };

    render(<SiteCard site={site} onEdit={jest.fn()} />);

    expect(screen.getByText('example.com')).toBeInTheDocument();
    expect(screen.getByText('PHP 8.2')).toBeInTheDocument();
  });

  it('calls onEdit when edit button is clicked', () => {
    const onEdit = jest.fn();
    render(<SiteCard site={site} onEdit={onEdit} />);

    fireEvent.click(screen.getByRole('button', { name: /edit/i }));

    expect(onEdit).toHaveBeenCalledWith(1);
  });
});

// Example: Hook test
describe('useMetrics', () => {
  it('fetches and updates metrics', async () => {
    const { result } = renderHook(() => useMetrics());

    // Initial state
    expect(result.current.loading).toBe(true);

    // Wait for data
    await waitFor(() => {
      expect(result.current.loading).toBe(false);
      expect(result.current.metrics).toBeDefined();
    });
  });
});
```

Slide 18: Monitoring & Observability

System Monitoring

Metrics Collection

```
â”Œâ”€â”€â”€â
```



Metrics API

```
// System metrics endpoint
type SystemMetrics struct {
    Timestamp time.Time  `json:"timestamp"`

    CPU       CPUMetrics  `json:"cpu"`
    Memory    MemMetrics  `json:"memory"`
    Disk      DiskMetrics `json:"disk"`
    Network   NetMetrics  `json:"network"`
    Services  []ServiceStatus `json:"services"`
    Uptime    time.Duration   `json:"uptime"`
}

type CPUMetrics struct {
    Percent    float64   `json:"percent"`
    User       float64   `json:"user"`
    System     float64   `json:"system"`
    Idle       float64   `json:"idle"`
    CoreCount  int       `json:"core_count"`
    PerCore    []float64 `json:"per_core"`
}

type MemMetrics struct {
    Total       uint64  `json:"total"`        // bytes
    Used        uint64  `json:"used"`
    Available   uint64  `json:"available"`
    Cached      uint64  `json:"cached"`
    UsedPercent float64 `json:"used_percent"`
}

type DiskMetrics struct {
    Partitions []Partition `json:"partitions"`
    IOStats    IOStats      `json:"io_stats"`
}

type NetMetrics struct {
    BytesSent   uint64    `json:"bytes_sent"`
    BytesRecv   uint64    `json:"bytes_recv"`
    PacketsSent uint64    `json:"packets_sent"`
    PacketsRecv uint64    `json:"packets_recv"`
    ErrIn      uint64    `json:"err_in"`
    ErrOut     uint64    `json:"err_out"`
    PerInterface []IFaceStats `json:"per_interface"`
}
```

Slide 19: Error Handling & Logging

Logging & Error Management

Structured Logging

```
// zerolog configuration
import "github.com/rs/zerolog"

func initLogger() zerolog.Logger {
    // Console writer with color
    output := zerolog.ConsoleWriter{Out: os.Stdout, TimeFormat: time.RFC3339}

    logger := zerolog.New(output).
        Level(zerolog.InfoLevel).
        With().
        Timestamp().
        Caller().
        Str("service", "itdlupanel").
        Logger()

    return logger
}

// Usage with structured logging
log.Info().
    Str("user", username).
    Str("action", "login").
    Str("ip", clientIP).
    Msg("User logged in")

log.Error().
    Err(err).
    Str("site_id", "123").
    Str("operation", "backup").
    Msg("Backup failed")
```

Error Types

```
// Custom error types
type AppError struct {
    Code    string `json:"code"`
    Message string `json:"message"`
    Details string `json:"details,omitempty"`
    Status  int    `json:"-"`
}

var (
    ErrNotFound = &AppError{
        Code:    "NOT_FOUND",
        Message: "Resource not found",
        Status:  404,
    }

    ErrUnauthorized = &AppError{
        Code:    "UNAUTHORIZED",
        Message: "Authentication required",
        Status:  401,
    }

    ErrForbidden = &AppError{
        Code:    "FORBIDDEN",
        Message: "Permission denied",
        Status:  403,
    }

    ErrValidation = &AppError{
        Code:    "VALIDATION_ERROR",
        Message: "Invalid request data",
        Status:  400,
    }

    ErrInternal = &AppError{
        Code:    "INTERNAL_ERROR",
```

```
        Message: "Internal server error",
        Status:  500,
    }
)

// Error handling middleware
func ErrorHandler() gin.HandlerFunc {
    return func(c *gin.Context) {
        c.Next()

        if len(c.Errors) > 0 {
            err := c.Errors.Last()
            appErr, ok := err.Err.(*AppError)
            if !ok {
                appErr = ErrInternal
            }

            c.JSON(appErr.Status, gin.H{
                "error": appErr.Code,
                "message": appErr.Message,
                "details": appErr.Details,
            })
        }
    }
}
```

Audit Log Format

```
{
  "id": 12345,
  "user_id": 1,
  "username": "admin",
  "action": "UPDATE",
  "resource": "sites",
  "details": {
    "site_id": 5,
    "domain": "example.com",
    "changes": {
      "php_version": {"old": "8.1", "new": "8.2"},
      "ssl_enabled": {"old": false, "new": true}
    }
  },
  "ip_address": "192.168.1.100",
  "user_agent": "Mozilla/5.0...",
  "created_at": "2026-06-30T14:25:03Z"
}
```

Slide 20: KáºiLuáºn & Roadmap

Tá»•ng Káº¿t

Nhá»¯ng Äiá»fm Ná»•i Báºt Cá»¹ Thuáºt

```
â”Œâ”€â”€â”€â
```

```
â”  â”     Local, FTP, S3, Google Drive, OneDrive + Encryption             â”  â”
â”  â”œâ”€â”€â”€â
```